Daily Problem: 19–Feb–2026

Problem Statement

Let Σ be a finite alphabet and consider the language

$$L \;=\; \{\, ww \mid w \in \Sigma^* \,\}.$$

Show that:

1. L can be accepted by a *non-deterministic* finite-state automaton equipped with a *queue*.
2. L cannot be accepted by any push-down automaton (PDA) with a *stack*.

Part 1: A nondeterministic queue automaton for $L = \{ww\}$

Queue automaton model

A *queue automaton* is a finite-state machine with a FIFO queue that supports:

- enqueue(x): add x to the rear,
- dequeue(): remove and return the front symbol,

and may make ε-moves. We assume nondeterminism in the control.

High-level idea

On input $x \in \Sigma^*$, we want to check whether x can be split into two halves $x = uv$ with $u = v$. A queue is perfect for this: we can store u by enqueuing its symbols, then compare v against the queue by dequeuing.

The only missing piece is: we do not know where the middle is. We use nondeterminism to guess the split point.

Construction

We describe an NFA-with-queue Q that recognizes L.

States (modes). The control has three modes:

- q_{push}: read symbols and enqueue them (building the first half w),
- q_{pop}: read symbols and compare by dequeuing (checking the second half matches),
- q_{acc}: accepting state.

Operation. On input x:

1. Start in q_{push} with an empty queue.
2. While in q_{push}, for each input symbol $a \in \Sigma$, nondeterministically choose one of:
   - *Keep pushing:* read a, enqueue a, stay in q_{push}.
   - *Guess the middle:* ε-move to q_{pop} (do not consume input).
3. While in q_{pop}, for each input symbol $a \in \Sigma$, do:
   - dequeue the front symbol b of the queue; if the queue is empty, reject,
   - if $a \neq b$, reject,
   - if $a = b$, consume a and stay in q_{pop}.
4. When the input is finished, accept iff the queue is empty (and then move to q_{acc}).

Correctness proof

($\Rightarrow$) If $x \in L$, then Q accepts x.

If $x \in L$, then $x = ww$ for some $w \in \Sigma^*$. Consider the computation branch where Q stays in q_{push} for the first $|w|$ symbols, enqueuing exactly w, and then takes the ε-transition to q_{pop}. Now Q reads the remaining $|w|$ symbols (which are again w), and for each symbol it dequeues the matching symbol. Since the second half is identical to the first, all comparisons succeed, and exactly $|w|$ dequeues empty the queue. At end of input the queue is empty, so Q accepts.

($\Leftarrow$) If Q accepts x, then $x \in L$.

Suppose Q accepts x. In the accepting branch, at some point Q switches from q_{push} to q_{pop}. Let u be the prefix read while in q_{push}, and v be the remaining suffix read in q_{pop}. Then $x = uv$, and the queue initially contains exactly the symbols of u in order.

In q_{pop}, the machine reads v and dequeues one symbol per input symbol, requiring equality at each position. Thus v must match u symbol-by-symbol, and also the queue must be empty exactly when the input ends. Therefore $|u| = |v|$ and $u = v$, so $x = uu \in L$.

Thus Q accepts exactly L.

Part 2: $L = \{ww\}$ is not accepted by any PDA (not context-free)

To show no PDA accepts L, it suffices to show L is not context-free, since PDAs recognize exactly the context-free languages.

We prove L is not context-free using the pumping lemma for context-free languages.

CFL pumping lemma

If L is context-free, then there exists $p \geq 1$ such that every $s \in L$ with $|s| \geq p$ can be written as

$$s = uvxyz$$

with

$$|vxy| \leq p, \qquad |vy| \geq 1,$$

and for all $k \geq 0$,

$$uv^k xy^k z \in L.$$

Apply the lemma to a specific string

Assume for contradiction that L is context-free, and let p be its pumping length.

Pick two distinct symbols $a, b \in \Sigma$ (if $|\Sigma| = 1$, then $L = \{a^{2n}\}$ is regular; the interesting case is $|\Sigma| \geq 2$). Consider the string

$$s \;=\; (a^p b^p)(a^p b^p) \;=\; a^p b^p a^p b^p.$$

Clearly $s \in L$, since it is ww with $w = a^p b^p$. Also $|s| = 4p \geq p$.

Write $s = uvxyz$ with $|vxy| \leq p$ and $|vy| \geq 1$.

Because $|vxy| \leq p$, the substring vxy lies entirely within one of the four blocks

$$a^p,\ b^p,\ a^p,\ b^p$$

or spans across at most one boundary between two adjacent blocks.

Pumping breaks the "double" structure

We now show that pumping produces a string not of the form tt.

Intuitively, s has a very rigid structure: its first half is $a^p b^p$ and its second half is exactly the same. If we modify (pump) a short substring v in one local region of s, we destroy this perfect equality between halves.

More formally, consider the midpoint of s, which is after the first $2p$ symbols. There are two cases:

Case 1: vxy is entirely in the first half $a^p b^p$.

Then pumping changes only the first half and leaves the second half unchanged. So for $k \neq 1$, the pumped string $uv^k xy^k z$ has its first half different from its second half, hence cannot equal tt for any t. Therefore it is not in L, contradicting the pumping lemma.

Case 2: vxy is entirely in the second half $a^p b^p$.

This is symmetric: pumping changes only the second half and leaves the first half unchanged. Again for $k \neq 1$, the two halves differ, so the pumped string is not in L, contradiction.

Case 3: vxy crosses the midpoint.

Since $|vxy| \leq p$, it cannot cover both an entire copy of a^p and an entire copy of b^p across the midpoint; it overlaps only a limited region around the midpoint, which lies in the middle two blocks $b^p a^p$.

Pump with $k = 0$. Then vy (nonempty) is deleted from the middle region, so the length of the first half decreases but the length of the second half decreases by a different amount (because the deletion is centered at the midpoint and affects one side more than the other), making the two halves have different lengths or different symbol patterns. In either case, the result cannot be of the form tt. Thus $uv^0xy^0z \notin L$, contradiction.

In all cases we contradict the pumping lemma for CFLs. Hence L is not context-free.

Conclusion of Part 2

Since L is not context-free, no PDA (with a stack) can accept it.

Final Conclusion

- A nondeterministic finite-state automaton with a queue accepts $L = \{ww\}$ by guessing the midpoint, enqueuing the first half, and dequeuing to match the second half.
- No PDA accepts L, because L is not context-free.